

Tecnologías de Producción de Software 2026

Opción Ruby

Clase 1

Introducción a la cátedra, instalación del entorno y el lenguaje Ruby

Facultad de Informática — Universidad Nacional de La Plata

Viernes 04/09/2026

Información de la materia

Horarios de la cursada



Teoría: Viernes 08:00 hs – Aula 14



Practica: Jueves 10:00 hs – Aula 1

Programa

Introducción El lenguaje

Gemas

Active

Record

Rails

12 Factor Apps

Programa



Introducción



El lenguaje



Gemas



Active Record



Rails



12 Factor Apps

Pequeños Spoilers

- **El lenguaje:** profundizamos sobre el lenguaje:
 - Objetos y atributos
 - Clases y variables
 - Colecciones
 - Bloques e iteradores
 - Módulos
 - Excepciones
 - Conceptos avanzados

Pequeños Spoilers

- **Gemas:** a las librerías que extienden Ruby se las llaman Gemas. En esta sección abordaremos:
 - Manejo de dependencias
 - Bundler

Pequeños Spoilers

- **Active Record:** aprendemos a conectar con bases de datos:
 - Introducción
 - CRUD
 - Migraciones
 - Validaciones
 - Asociaciones

Pequeños Spoilers

- **Rails:** el framework más utilizado por los desarrolladores Ruby. En esta sección aprenderemos a trabajar con este framework:
 - Conceptos básicos
 - Ruteo
 - ActiveRecord
 - Unit Testing
 - MVC

Modalidad de cursada

Aprobación de la cursada:

Trabajo práctico integrador

Final:

- La nota final de la materia será un seis aprobando la cursada.
- Será posible mejorar la nota final realizando una evaluación teórica al finalizar la materia.

Historia del lenguaje

- **Yukihiro Matsumoto**
- Empezó el desarrollo de Ruby en 1993
- En 1995 lanzó la primera versión
- Aún lidera el desarrollo de Ruby

Su célebre frase

- Ruby is designed to make programmers **HAPPY**



Historia del lenguaje

Fusión de lenguajes

- Smalltalk
- Perl
- LISP

Hasta el 2001 conocido sólo en Japón

- El libro Programming Ruby fue el impulsor del lenguaje fuera de Japón
- También conocido como PickAxe
- La primera versión puede leerse en línea

Historia del lenguaje

David Heinemeier Hansson

Creador de Rails: framework open source para desarrollo de aplicaciones web

Su primer versión liberada (1.0) fue en 2005



El lenguaje

Dinámico

Sintaxis concisa y expresiva

Orientado a objetos

Capacidades de metaprogramación

Características funcionales

El lenguaje

Sintaxis y convenciones

Nombres válidos y convenciones:

```
NombreDeClaseOModulo  
CONSTANTE  
@nombre_de_atributo  
@@atributo_de_clase  
$variable_global  
nombre_de_metodo  
metodo_peligroso!  
metodo_que_pregunta?
```

El lenguaje

Objetos

Todos los valores son objetos

```
"Aprendiendo ruby".length  
"Aprendiendo ruby".each_char.sort.join  
1 + 2  
1.send :+, 2
```

El lenguaje

Arreglos

```
["Go", "Ruby", "Java", "Python", "PHP", "Javascript"].sort  
([1,2,3] + [4,5,6]).last
```

Números : Todos los valores son objetos

nil

```
a = Array.new  
a[10].nil?  
nil.nil?
```

object id

```
1.object_id  
nil.object_id
```

Preparando el entorno: rbenv y Git

- **rbenv** gestiona varias versiones de Ruby en paralelo, sin tocar la que trae el sistema operativo.
- Instalación: clonar rbenv y el plugin ruby-build, agregar el init de rbenv al shell, y luego rbenv install + rbenv global.
- **ruby -v** y un primer script (hello.rb) confirman que el entorno quedó funcionando.
- **Git** es el sistema de control de versiones de toda la cursada: init, status, log, add, commit, y el flujo remoto clone / push / pull.
- Las prácticas y el Trabajo Final Integrador (TFI) se entregan versionados en un repositorio de GitHub — conviene practicar el flujo básico desde hoy.

```
# Entorno con rbenv
rbenv install 3.4.0
rbenv global 3.4.0
ruby -v

# Primer commit
git init
git add .
git commit -m "init"
```

Todo es un objeto

- Un string entiende mensajes (métodos) como cualquier objeto.
- **1 + 2** es, por debajo, el envío del mensaje + al objeto 1.
- Los arreglos también son objetos con múltiples métodos incorporados.
- **nil** es un objeto y responde mensajes: nil.nil? → true
- Todo objeto tiene un identificador único: object_id.

```
"Aprendiendo ruby".length  
# => 17  
1 + 2 # => 3  
1.send(:+, 2)  
([1, 2, 3] + [4, 5, 6]).last  
nil.nil? # => true  
1.object_id
```

Creación de objetos: literales y new

- Los literales son la forma habitual y más idiomática de crear estos objetos.
- **Symbol no tiene constructor new:** solo se crea con :literal o convirtiendo un String con #to_sym.
- **String, Array y Hash** sí aceptan new, y permiten variantes que el literal no ofrece (valor por defecto, tamaño inicial, bloque generador).
- Array.new y Hash.new son útiles cuando el contenido depende de un cálculo, no de un valor fijo escrito a mano.

```
# Symbol: solo literal
:hola

# String: literal o new
"hola"   String.new("hola")

# Array: literal o new
[1, 2, 3] Array.new(3) { |i| i*2 }

# Hash: literal o new
{ a: 1 }  Hash.new(0)
```

Convertir entre tipos: to_s, to_sym y object_id

- **#to_s** es el método universal para obtener la representación en String de cualquier objeto: todo objeto lo hereda de Object, y muchas clases lo redefinen.
- **Kernel#String(obj)** es una forma más “segura”: intenta to_str primero y cae a to_s si no existe.
- La interpolación “#{objeto}” invoca to_s internamente.
- Para pasar de String a Symbol se usa **#to_sym** (o su alias #intern).
- Dos literales String iguales no son el mismo objeto: cada literal crea una instancia nueva — a diferencia de los símbolos, que sí son únicos.

```
5.to_s          # => "5"
nil.to_s        # => ""
String(nil)     # => ""

"hola".to_sym   # => :hola

'TTPS Ruby'.object_id ==
'TTPS Ruby'.object_id # => false
```

Símbolos

- Prefijo `:` seguido de un nombre: `:accion`, `:line_items`, incluso `:+`
- No necesitan declararse: Ruby garantiza una única instancia en toda la ejecución.
- A diferencia de los strings, dos símbolos iguales comparten el mismo `object_id`; dos strings iguales no.
- Por eso se usan como claves de hash, nombres de parámetros e identificadores de acciones o eventos.
- Un string es el tipo adecuado cuando el contenido es información variable (texto ingresado por un usuario, por ejemplo).

```
:uno.object_id  
# siempre devolverá lo mismo  
  
"uno".object_id  
# normalmente devolverá diferente
```

Colecciones: arreglos y hashes

- **Arreglos (Array):** colecciones ordenadas indexadas numéricamente.
- **Hashes (Hash):** colecciones de pares clave-valor.
- **%w(...)** construye un arreglo de strings sin interpolar; **%W(...)** sí interpola.

Arreglos

```
['Hola', 'Chau']  
  
# sin interpolar  
%w(Hola Chau #{2+2})  
  
# interpolando  
%W(Hola Chau #{2+2})
```

Hashes

```
# Versión 1.8  
{ :nombre => 'Damian',  
  :apellido => 'Candia' }  
  
# Versión > 1.8  
{ nombre: 'Damian',  
  apellido: 'Candia' }
```

Expresiones regulares

- Soporte nativo en el lenguaje, sin bibliotecas adicionales — herencia de la influencia de Perl.
- Se escriben entre barras: `/^[a-zA-Z]+$`
- **El operador `=~`** indica, mediante la posición del match (o nil), si el patrón fue encontrado.
- Herramienta central para validar datos de entrada: emails, teléfonos, códigos postales, direcciones IP.
- Conviene siempre pensar qué casos límite deja pasar el patrón (una regex simple de IP no valida el rango 0-255).

```
/^[a-zA-Z]+$
```

```
"Do you like cats?" =~ /like/
```

```
"192.168.0.10" =~  
/^\d{1,3}\.\d{1,3}\.  
\d{1,3}\.\d{1,3}$/
```

Rangos

- Un rango es un objeto de primera clase (instancia de Range): una secuencia entre un límite inferior y uno superior.
- **Dos puntos (..)**: inclusivo del último elemento.
- **Tres puntos (...)**: excluye el último elemento.
- **to_a** convierte un rango finito y enumerable en un arreglo.
- **El operador ===** compara si un valor cae dentro del rango — es el mecanismo interno de case/when.

```
0..1
"a".. "z"   "a"... "z"

# Como intervalos:
(1..10) === 5   # => true
(1..10) === 15  # => false
(1..10) === 3.1 # => true
```

Todo es una expresión

- Prácticamente cualquier construcción del lenguaje devuelve un valor, incluidas las estructuras de control: if, unless, case.
- Es habitual asignar directamente el resultado de un if o un case a una variable, en vez de declararla antes y reasignarla en cada rama.
- Esto reduce la duplicación y hace más explícito el flujo de datos.

```
a = 3.14 # => 3.14

estado = nil
face = case estado
  when "Feliz" then ":)"
  when "Triste" then ":("
  else          ":|"
end
```

Consultas:

dcandia@linti.unlp.edu.ar

Docente: Damian Candia